

作业一：文本分类实现

Bag of Words、Word2Vec 与 BERT 三种文本表示方法的对比研究

姓名：郝普瑞 学号：2310412

2026 年 10 月 9 日

摘要

本报告在 NYT 三分类新闻数据集上，系统比较了三类文本表示方法：基于词袋的离散表示（Binary BoW / Word Frequency）、基于词向量的稠密表示（预训练 GloVe、自训练 Word2Vec）、以及基于预训练语言模型的上下文表示（BERT-base-uncased 微调）。所有实验共用同一份 80%/10%/10% 数据划分，分类器统一为 Logistic Regression（BERT 除外），评价指标统一为 Accuracy 与 Macro-F1。

主要发现：（1）词袋表示在该数据集上异常强，词频词袋的 Test Macro-F1 达 0.9542，优于随机初始化的词向量平均；（2）词向量的价值主要体现在压缩比——100 维 GloVe 追平了 53146 维的 Binary BoW (0.9471 vs 0.9468)，维度压缩约 500 倍而性能几乎不变；（3）BERT 微调取得最优结果但优势有限——4 次独立运行的 Test Macro-F1 为 0.9597 ± 0.0033 ，平均仅比最强词袋高 0.0055，且受 GPU 浮点非确定性影响，单次结果不可复现；（4）错误高度集中在 business ↔ politics 之间（占全部错误的 75%），而 sports 类的错分率仅 0.35%。

目录

1	实验目的与任务概述	3
2	数据集与统一实验设置	4
2.1	数据集与统计	4
2.2	数据划分	4
2.3	评价指标	5
2.4	实验环境	5
3	Task 1：基于词袋表示的文本分类	5
3.1	方法原理	5
3.2	实现细节	6

目录	2
3.3 实验结果	6
3.4 结果分析	6
4 Task 2: 基于词向量的文本分类	9
4.1 方法原理	9
4.2 三组实验设置	9
4.3 实验结果	10
4.4 结果分析	13
5 Task 3: 基于 BERT 微调的文本分类	13
5.1 方法原理	13
5.2 实验设置	14
5.3 截断分析	14
5.4 实验结果	15
5.5 结果分析	15
5.6 多次运行与可复现性	19
6 综合对比与讨论	19
6.1 三种文本表示的横向对比	19
6.2 错误分析	20
6.3 结论与局限性	21
A 运行环境与复现步骤	22
B 问题排查记录	23

1 实验目的与任务概述

本实验旨在理解并比较不同文本表示方法在文本分类任务中的原理与效果，具体包括四个目标：掌握词袋表示与 Logistic Regression 的基本用法；掌握 Word2Vec / GloVe 词向量的获取与使用；熟悉 BERT 类预训练语言模型的微调流程；并能够用 Accuracy 与 Macro-F1 对模型做出评价与横向比较。

作业包含三个任务：

- **Task 1 (Bag of Words):** 用 Binary BoW 与 Word Frequency 两种词袋表示，分别训练 Logistic Regression 分类器；
- **Task 2 (Word2Vec):** 统一使用 100 维词向量，分别来自预训练 GloVe、AG News 自训练 Word2Vec、NYT 自训练 Word2Vec，文档向量取文档内有效词向量的平均，再用 Logistic Regression 分类；
- **Task 3 (预训练语言模型) :** 微调 google-bert/bert-base-uncased，设置 `max_length = 64`、训练 3 epochs。

表 1 对照列出作业的硬性要求与本实验的实现方式。

表 1: 作业要求与本实验实现的对照

作业要求	本实验实现
Task 1: 两种 BoW 表示 + LR, 报告 Acc 与 Macro-F1	Binary BoW 与 Word Frequency 各一组，另加两组 <code>class_weight=balanced</code> 对照
Task 2: 统一 100 维词向量，文档向量为词向量平均，分类器为 LR	GloVe-100 / Word2Vec(AG News)-100 / Word2Vec(NYT)-100 三组， $e(d) = \frac{1}{n} \sum_{i=1}^n e(w_i)$ ，LR 分类
Task 3: bert-base-uncased, <code>max_length=64</code> , 3 epochs	该模型、该长度、该轮数，严格遵守
NYT 随机打乱后按 80%/10%/10% 划分，所有实验使用同一划分	由 <code>code/step0_split.py</code> 一次性生成并落盘， <code>seed=42</code> ，六个实验共用 <code>data/{train,val,test}.csv</code>
统一在 NYT Test Set 上评价，报告 Accuracy 与 Macro-F1	全部结果均在 1145 条 Test 样本上给出，指标同时报告

2 数据集与统一实验设置

2.1 数据集与统计

实验主数据集为 NYT 新闻数据 (HW-1/nyt.csv, 字段为 text 与 label), 辅助数据集为 AG News (HW-1/ag.csv, 仅提供 text, 用于训练 Word2Vec)。清洗方式为去除空值并按 text 去重。统计结果见表 2。

表 2: NYT 数据集统计 (清洗后)

项目	值	说明
总样本数	11447	—
类别数	3	business / politics / sports
sports	8597 (75.1%)	绝对多数类
politics	1433 (12.5%)	—
business	1417 (12.4%)	—
平均文本长度	637.1 词	按空格切分粗略统计
平均 subword token 数	834.2	中位数 872 / P90 1239 / 最长 6775
超过 64 token 的文档占比	99.96%	截断后平均保留 7.67%
Training / Validation / Test	9157 / 1145 / 1145	80% / 10% / 10%

有两个必须在报告中讨论的数据特点:

(1) **类别严重不均衡**。sports 占 75.1%, 因此 Accuracy 存在约 75% 的“地板”: 一个永远输出 sports 的平凡基线即可获得 0.7511 的 Accuracy (实测, 见表 9)。这直接说明**必须同时考察 Macro-F1**——该基线的 Macro-F1 仅为 0.2860。这也是作业要求同时报告两个指标的动机。

(2) **文本极长**, 而作业规定 BERT 的 `max_length=64`。实测平均 834.2 个 subword token (中位数 872, P90 1239, 最长 6775), **99.96% 的文档会被截断**, 截断后平均仅保留 **7.67%** 的 token。即 BERT 只能看到每篇文档开头约 7.7% 的内容。该参数是作业硬性规定, 必须遵守, 但需在第 5.3 节量化其影响。

2.2 数据划分

按作业要求, 先对 NYT 数据集做随机打乱, 再按 80%/10%/10% 划分为 Training / Validation / Test (`seed=42`, 且按类别分层抽样以保证三份数据的类别比例一致)。划分结果由 `code/step0_split.py --stratify` 落盘到 `data/`, 后续所有任务均读取这三份文件, 从而保证不同方法之间的可比性——这是跨任务比较的前提。

需要说明的是: 词表与文档向量统计均只在训练集上拟合。Task 1 的词表仅用训练集 `fit`, `val/test` 只做 `transform`; Task 2 中 NYT 自训练 Word2Vec 也只使用训练集的 9157 篇文本, 以避免验证/测试集的信息泄漏。

2.3 评价指标

Accuracy 为分类正确的样本数占全部测试样本的比例：

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\hat{y}_i = y_i] \quad (1)$$

Macro-F1 先对每个类别分别计算 F1，再做算术平均（各类权重相同）：

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C \frac{2P_c R_c}{P_c + R_c}, \quad P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c} \quad (2)$$

其中 $C = 3$ 为类别数。由于各类权重相同，Macro-F1 对少数类的表现更敏感，在类别不均衡的数据上比 Accuracy 更能反映真实分类能力。

2.4 实验环境

表 3: 实验环境

项目	版本 / 配置
操作系统	Windows
GPU	NVIDIA GeForce RTX 4060 Laptop GPU (8 GB 显存)
显卡驱动	566.26 (支持 CUDA ≤ 12.7)
Python	3.11.17 (conda 环境)
torch	2.6.0+cu124
transformers	5.19.0
datasets / accelerate	5.1.0 / 1.15.0
gensim	4.4.0
scikit-learn	1.9.1
numpy / pandas	2.4.6 / 3.0.6
nlTK	3.10.3

`torch.cuda.is_available()` 验证结果为 True，BERT 训练使用 bf16 混合精度。

3 Task 1: 基于词袋表示的文本分类

3.1 方法原理

设训练语料构建的词表大小为 $|V|$ ，则每篇文档被表示为 $|V|$ 维向量。

Binary Bag of Words 只记录词是否出现：

$$x_i = \begin{cases} 1, & \text{词表中第 } i \text{ 个词出现在文档中} \\ 0, & \text{否则} \end{cases} \quad (3)$$

Word Frequency 记录词在当前文档中的出现次数:

$$x = (c_1, c_2, \dots, c_{|V|}), \quad c_i = \text{第 } i \text{ 个词的出现次数} \quad (4)$$

两种表示得到的高维稀疏向量直接作为 Logistic Regression 的输入特征。

3.2 实现细节

- **分词:** 统一使用小写化 + `nlk word_tokenize` (`code/common.py` 中的 `tokenize`), 保证三个任务的分词口径一致;
- **词表:** `CountVectorizer`, 设置 `min_df=2` 丢弃出现次数少于 2 的极低频词, 且仅在训练集上 `fit`;
- **分类器:** `LogisticRegression(max_iter=1000, C=1.0)`, 默认 `lbfgs` 求解器;
- **对照实验:** 额外运行 `class_weight="balanced"` 的两组, 考察类别不均衡的影响;
- **基线:** 同时打印“永远预测多数类”的平凡基线, 作为参照系。

3.3 实验结果

表 4: Task 1 结果 (NYT Test Set, 1145 条)

方法	词表 V	Val Acc	Val Macro-F1	Test Acc	Test Macro-F1
多数类基线	—	—	—	0.7511	0.2860
Binary BoW + LR	53146	0.9773	0.9436	0.9790	0.9468
Word Frequency + LR	53146	0.9764	0.9449	0.9825	0.9542
Binary BoW + LR (balanced)	53146	0.9808	0.9514	0.9790	0.9479
Word Frequency + LR (balanced)	53146	0.9790	0.9485	0.9808	0.9506

图 1 与图 2 为两组主实验的终端输出, 图 3 与图 4 为两组类别权重对照实验的输出。

3.4 结果分析

(1) 两种词袋都极强, 且远好于平凡基线。Test Accuracy 在 0.979~0.983 之间, Macro-F1 在 0.947~0.954 之间。相比多数类基线的 Macro-F1 0.2860, 提升约 +**0.67**, 说明模型确实学到了区分性特征, 而不是靠猜多数类。

```

PS C:\Users\jmlcsr\Desktop\lab1> cd c:\Users\jmlcsr\Desktop\lab1; & 'D:\Miniconda\envs\lab1\python.exe' code\
task1_bow.py --mode binary
[binary] 词表大小 |V| = 53146, 训练集矩阵维度 = (9157, 53146)
[binary] 稀疏矩阵非零元素占比 = 0.005935
[多数类基线(全预测 sports)] Accuracy=0.7511 Macro-F1=0.2860
[Binary-Bow + LR (val)] Accuracy=0.9773 Macro-F1=0.9436
      precision    recall  f1-score   support

   business      0.9394      0.8794      0.9084        141
   politics      0.9067      0.9444      0.9252        144
    sports      0.9954      0.9988      0.9971        860

   accuracy                0.9773        1145
  macro avg      0.9471      0.9409      0.9436        1145
 weighted avg      0.9773      0.9773      0.9771        1145

[Binary-Bow + LR] Accuracy=0.9790 Macro-F1=0.9468
      precision    recall  f1-score   support

   business      0.9545      0.8873      0.9197        142
   politics      0.8896      0.9580      0.9226        143
    sports      0.9988      0.9977      0.9983        860

   accuracy                0.9790        1145
  macro avg      0.9477      0.9477      0.9468        1145
 weighted avg      0.9797      0.9790      0.9791        1145

结果已记录到 results\results.json

```

图 1: Binary Bag of Words + LR 的运行输出。可见词表规模 $|V| = 53146$ 、稀疏矩阵非零元素占比 0.5935%、多数类基线 Acc 0.7511 / Macro-F1 0.2860，最终 Test Acc 0.9790 / Macro-F1 0.9468。

```

PS C:\Users\jmlcsr\Desktop\lab1> & 'D:\Miniconda\envs\lab1\python.exe' code\task1_bow.py --mode freq
[freq] 词表大小 |V| = 53146, 训练集矩阵维度 = (9157, 53146)
[freq] 稀疏矩阵非零元素占比 = 0.005935
D:\Miniconda\envs\lab1\Lib\site-packages\sklearn\linear_model\logistic.py:1435: FutureWarning: 'n_jobs' has
no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
[多数类基线(全预测 sports)] Accuracy=0.7511 Macro-F1=0.2860
[WordFrequency + LR (val)] Accuracy=0.9764 Macro-F1=0.9449
      precision    recall  f1-score   support

   business      0.9338      0.9007      0.9170        141
   politics      0.9296      0.9167      0.9231        144
    sports      0.9908      0.9988      0.9948        860

   accuracy                0.9764        1145
  macro avg      0.9514      0.9387      0.9449        1145
 weighted avg      0.9761      0.9764      0.9762        1145

[WordFrequency + LR] Accuracy=0.9825 Macro-F1=0.9542
      precision    recall  f1-score   support

   business      0.9420      0.9155      0.9286        142
   politics      0.9189      0.9510      0.9347        143
    sports      1.0000      0.9988      0.9994        860

   accuracy                0.9825        1145
  macro avg      0.9536      0.9551      0.9542        1145
 weighted avg      0.9827      0.9825      0.9826        1145

结果已记录到 results\results.json
PS C:\Users\jmlcsr\Desktop\lab1> & 'D:\Miniconda\envs\lab1\python.exe' code\task1_bow.py --mode binary --clas
s weight balanced

```

图 2: Word Frequency + LR 的运行输出。词表规模与 Binary 完全相同 (53146)，最终 Test Acc 0.9825 / Macro-F1 0.9542，优于二值表示。

```
[binary] 词表大小 |V| = 53146, 训练集矩阵维度 = (9157, 53146)
[binary] 稀疏矩阵非零元素占比 = 0.005935
D:\Miniconda\envs\lab1\Lib\site-packages\sklearn\linear_model\_logistic.py:1435: FutureWarning: 'n_jobs' has
no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
[多数类基线(全预测 sports)] Accuracy=0.7511 Macro-F1=0.2860
[Binary-Bow + LR [class_weight=balanced] (val)] Accuracy=0.9808 Macro-F1=0.9514
```

	precision	recall	f1-score	support
business	0.9343	0.9078	0.9209	141
politics	0.9195	0.9514	0.9352	144
sports	0.9988	0.9977	0.9983	860
accuracy			0.9808	1145
macro avg	0.9509	0.9523	0.9514	1145
weighted avg	0.9809	0.9808	0.9808	1145

```
[Binary-Bow + LR [class_weight=balanced]] Accuracy=0.9790 Macro-F1=0.9479
```

	precision	recall	f1-score	support
business	0.9618	0.8873	0.9231	142
politics	0.8846	0.9650	0.9231	143
sports	0.9988	0.9965	0.9977	860
accuracy			0.9790	1145
macro avg	0.9484	0.9496	0.9479	1145
weighted avg	0.9800	0.9790	0.9791	1145

结果已记录到 results\results.json

图 3: Binary BoW + LR (class_weight="balanced") 的输出。Test Acc 0.9790 / Macro-F1 0.9479, 与不加权基本持平。

```
PS C:\Users\jmlcsr\Desktop\lab1> & 'D:\Miniconda\envs\lab1\python.exe' code\task1_bow.py --mode freq --class_
weight balanced
[freq] 词表大小 |V| = 53146, 训练集矩阵维度 = (9157, 53146)
[freq] 稀疏矩阵非零元素占比 = 0.005935
D:\Miniconda\envs\lab1\Lib\site-packages\sklearn\linear_model\_logistic.py:1435: FutureWarning: 'n_jobs' has
no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
[多数类基线(全预测 sports)] Accuracy=0.7511 Macro-F1=0.2860
[WordFrequency + LR [class_weight=balanced] (val)] Accuracy=0.9790 Macro-F1=0.9485
```

	precision	recall	f1-score	support
business	0.9270	0.9007	0.9137	141
politics	0.9252	0.9444	0.9347	144
sports	0.9965	0.9977	0.9971	860
accuracy			0.9790	1145
macro avg	0.9496	0.9476	0.9485	1145
weighted avg	0.9790	0.9790	0.9790	1145

```
[WordFrequency + LR [class_weight=balanced]] Accuracy=0.9808 Macro-F1=0.9506
```

	precision	recall	f1-score	support
business	0.9353	0.9155	0.9253	142
politics	0.9122	0.9441	0.9278	143
sports	1.0000	0.9977	0.9988	860
accuracy			0.9808	1145
macro avg	0.9491	0.9524	0.9506	1145
weighted avg	0.9810	0.9808	0.9808	1145

结果已记录到 results\results.json

图 4: Word Frequency + LR (class_weight="balanced") 的输出。Test Acc 0.9808 / Macro-F1 0.9506, 反而略低于不加权版本。

(2) **Word Frequency 优于 Binary BoW**。Test Macro-F1 为 0.9542 vs 0.9468 (+0.0074), Accuracy 为 0.9825 vs 0.9790 (+0.0035)。两组实验的维度完全相同 ($|V| = 53146$)，唯一的差别就是**是否保留词频**，因此该差异可归因于词频携带了“强度”信息——某类新闻的特征词往往在文中反复出现。

(3) **class_weight="balanced" 没有系统性收益**。Binary 从 0.9468 变为 0.9479 (+0.0011), Frequency 从 0.9542 变为 0.9506 (−0.0036)，两者都在随机波动范围内。原因在于：该数据集虽然类别不均衡，但**类别可分性极强** (sports 的特征词极为显著)，默认 LR 已能把少数类学好；balanced 只是把少数类 recall 拉高、precision 拉低，Macro-F1 因此打平。

(4) **高维稀疏的具体形态**。词表共 53146 个词，训练矩阵为 9157×53146 (约 4.87 亿个元素)，非零元素仅占 **0.5935%** (稀疏度 99.41%)，等价于每篇文档平均只用到约 315 个不同的词 (而全文平均 637 词)。这是“词表大、单篇用词集中”的直接结果。需要说明的是，非零占比只是表示的形态描述，**不是评价指标**。

(5) **val 与 test 高度一致** (如 Binary: 0.9436 vs 0.9468)，没有明显过拟合，test 略优于 val 属于抽样波动。

(6) **该实验是确定性的**。在去掉 sklearn 已忽略的 n_jobs 参数后重跑 --mode binary，输出与修改前**逐位完全一致** (Val 0.9773/0.9436、Test 0.9790/0.9468, $|V|$ 与非零占比亦相同)。这说明 Task 1 无随机成分 (CountVectorizer 是确定性的，lbfgs 求解器无随机初始化)，重复运行只会得到完全相同的结果，因此无需多次实验。

4 Task 2: 基于词向量的文本分类

4.1 方法原理

统一使用 100 维词向量。对一篇包含 n 个有效词 (能在词向量表中查到) 的文档，其文档向量为所有有效词向量的平均：

$$e(d) = \frac{1}{n} \sum_{i=1}^n e(w_i) \quad (5)$$

该表示把任意长度的文档压缩为固定 100 维的稠密向量，再交给 Logistic Regression 分类。若文档中不存在任何有效词，则退回零向量 (本实验中该情况出现 0 次)。

三组实验的唯一变量是**词向量的来源**：预训练 GloVe (60 亿词语料)、AG News 自训练、NYT 自训练。

4.2 三组实验设置

- **GloVe-100**: 官方 glove.6B.100d.txt (347,117,594 字节, 已核对与官方一致), 词表 400,001 词;

- **Word2Vec(AG News):** 在 AG News 的 90,000 条文本上训练，语料共 4,051,013 词（平均每篇仅 45 词），词表 25,376；
- **Word2Vec(NYT):** 仅用训练集的 9,157 篇文本训练（避免 val/test 信息泄漏），语料共 6,676,990 词（平均每篇 729 词），词表 31,579。

Word2Vec 训练超参数统一为 `vector_size=100`、`window=5`、`min_count=5`、`sg=1`、`negative=5`、`epochs=10`、`seed=42`（Skip-gram + 负采样）。分类器统一为 `LogisticRegression(max_iter=2000)`。

4.3 实验结果

表 5: Task 2 结果（NYT Test Set, 1145 条）

词向量来源	训练语料	词表	词覆盖率 (test)	Test Acc	Test Macro-F1
GloVe-100（预训练）	60 亿词	400001	98.06%	0.9782	0.9471
Word2Vec(NYT)-100	6.68M 词（同领域）	31579	97.58%	0.9712	0.9308
Word2Vec(AG News)-100	4.05M 词（跨领域）	25376	94.32%	0.9686	0.9227

图 5、图 6、图 7 分别为三组实验的终端输出。

```
PS C:\Users\jmlcsr\Desktop\lab1> cd c:\Users\jmlcsr\Desktop\lab1; & 'D:\Miniconda\envs\lab1\python.exe' code\
task2_word2vec.py --source glove; & 'D:\Miniconda\envs\lab1\python.exe' code\task2_word2vec.py --source ag;
'D:\Miniconda\envs\lab1\python.exe' code\task2_word2vec.py --source nyt
GloVe 词表大小: 400001
[glove:train] 词覆盖率 0.9807 | 无有效词文档 0/9157
[glove:val] 词覆盖率 0.9804 | 无有效词文档 0/1145
[glove:test] 词覆盖率 0.9806 | 无有效词文档 0/1145
[GloVe-100 mean + LR (val)] Accuracy=0.9747 Macro-F1=0.9364
      precision    recall  f1-score   support

    business      0.9380      0.8582      0.8963        141
    politics      0.8889      0.9444      0.9158        144
     sports      0.9954      0.9988      0.9971        860

   accuracy              0.9747        1145
  macro avg      0.9407      0.9338      0.9364        1145
 weighted avg      0.9749      0.9747      0.9745        1145

[GloVe-100 mean + LR] Accuracy=0.9782 Macro-F1=0.9471
      precision    recall  f1-score   support

    business      0.9545      0.8873      0.9197        142
    politics      0.9007      0.9510      0.9252        143
     sports      0.9954      0.9977      0.9965        860

   accuracy              0.9782        1145
  macro avg      0.9502      0.9453      0.9471        1145
 weighted avg      0.9785      0.9782      0.9781        1145

结果已记录到 results\results.json
```

图 5: 预训练 GloVe-100 平均词向量 + LR 的输出。词表 400,001，词覆盖率 98.07%，Test Acc 0.9782 / Macro-F1 0.9471。

```

AG News 文本数: 90000
训练语料句子数: 90000, 总词数: 4051013
Word2Vec 词表大小: 25376
[ag:train] 词覆盖率 0.9425 | 无有效词文档 0/9157
[ag:val] 词覆盖率 0.9423 | 无有效词文档 0/1145
[ag:test] 词覆盖率 0.9432 | 无有效词文档 0/1145
[W2V(AGNews)-100 mean + LR (val)] Accuracy=0.9677 Macro-F1=0.9233

```

	precision	recall	f1-score	support
business	0.9213	0.8298	0.8731	141
politics	0.8710	0.9375	0.9030	144
sports	0.9919	0.9953	0.9936	860
accuracy			0.9677	1145
macro avg	0.9280	0.9209	0.9233	1145
weighted avg	0.9680	0.9677	0.9674	1145

```

[W2V(AGNews)-100 mean + LR] Accuracy=0.9686 Macro-F1=0.9227

```

	precision	recall	f1-score	support
business	0.9037	0.8592	0.8809	142
politics	0.8675	0.9161	0.8912	143
sports	0.9965	0.9953	0.9959	860
accuracy			0.9686	1145
macro avg	0.9226	0.9235	0.9227	1145
weighted avg	0.9689	0.9686	0.9686	1145

```

结果已记录到 results\results.json

```

图 6: Word2Vec(AG News)-100 平均词向量 + LR 的输出。训练语料 4,051,013 词，词表 25,376，词覆盖率 94.32%，Test Acc 0.9686 / Macro-F1 0.9227。

```

训练语料句子数: 9157, 总词数: 6676990
Word2Vec 词表大小: 31579
[nyt:train] 词覆盖率 0.9796 | 无有效词文档 0/9157
[nyt:val] 词覆盖率 0.9753 | 无有效词文档 0/1145
[nyt:test] 词覆盖率 0.9758 | 无有效词文档 0/1145
[W2V(NYT)-100 mean + LR (val)] Accuracy=0.9668 Macro-F1=0.9205
      precision    recall  f1-score   support

   business      0.9141      0.8298      0.8699       141
   politics      0.8800      0.9167      0.8980       144
    sports      0.9896      0.9977      0.9936       860

   accuracy                    0.9668       1145
  macro avg      0.9279      0.9147      0.9205       1145
 weighted avg      0.9665      0.9668      0.9664       1145

[W2V(NYT)-100 mean + LR] Accuracy=0.9712 Macro-F1=0.9308
      precision    recall  f1-score   support

   business      0.9242      0.8592      0.8905       142
   politics      0.8919      0.9231      0.9072       143
    sports      0.9919      0.9977      0.9948       860

   accuracy                    0.9712       1145
  macro avg      0.9360      0.9266      0.9308       1145
 weighted avg      0.9710      0.9712      0.9709       1145

结果已记录到 results\results.json
PS C:\Users\jmlcsr\Desktop\lab1>

```

图 7: Word2Vec(NYT)-100 平均词向量 + LR 的输出。仅用训练集 9,157 篇文本训练, 词表 31,579, 词覆盖率 97.58%, Test Acc 0.9712 / Macro-F1 0.9308。

4.4 结果分析

(1) 核心规律：词覆盖率与 Macro-F1 完全同序。

覆盖率：98.06% > 97.58% > 94.32% $\iff$ Macro-F1: 0.9471 > 0.9308 > 0.9227

三组实验的唯一差别是词向量来源，因此覆盖率能够完整解释性能排序。其机理是：覆盖率低 $\Rightarrow$ 更多词查不到词向量 $\Rightarrow$ 文档平均向量被“稀释”（分母 n 变小但有效信息更少）、判别信息减少。需要说明的是，三组的“无有效词文档数”均为 0，即不存在被喂入全零向量的文档，性能差异来自覆盖率的精细差别而非极端情形。

(2) 同领域自训练优于跨领域自训练。W2V(NYT) 的 Macro-F1 为 0.9308，高于 W2V(AG News) 的 0.9227 (+0.0081)，原因有二：语料与测试集同领域（都是 NYT 新闻），且覆盖率更高（97.58% vs 94.32%）。AG News 文本极短（平均 45 词），可供模型学习的上下文共现信息也有限。

(3) 自训练词向量仍明显落后于预训练 GloVe。W2V(NYT) 比 GloVe 低 0.0163 (0.9308 vs 0.9471)。决定性因素是语料规模的数量级差距：自训练语料仅 668 万词，而 GloVe 基于 60 亿词，相差近 900 倍。

(4) 与 Task 1 的关系（重要）：“压缩比”才是词向量的核心贡献。GloVe 以 100 维追平了 53146 维的 Binary BoW (0.9471 vs 0.9468，仅差 0.0003)，即约 500 倍维度压缩而性能几乎无损——说明“词向量平均”在高维稀疏 $\rightarrow$ 低维稠密的转换上是成功的。但它并未超越词频词袋（低 0.0071）。这与数据集特点有关：文本极长（平均 637 词），词袋能无损保留全部词汇信息，而平均词向量把 637 个词压成 100 维，信息损失较大；同时平均操作丢弃了语序，“狗咬人”与“人咬狗”不可区分。

5 Task 3: 基于 BERT 微调的文本分类

5.1 方法原理

BERT 是双向 Transformer 编码器，在大规模语料上通过掩码语言模型等目标预训练，能够得到上下文相关的词表示。用于文本分类时，在 [CLS] 位置的输出（经 pooler）之上接一个线性分类头，然后用下游数据微调整个网络。

与 Task 2 的“平均词向量”相比，其关键差异是：

- 表示是上下文相关的，同一个词在不同语境下得到不同向量；
- 表示是端到端学习的，不依赖预先固定的词向量表；
- 因此不需要手动词表与 OOV 处理，理论上能更好地刻画长距离依赖。

表 6: Task 3 超参数

参数	值
预训练模型	google-bert/bert-base-uncased
max_length	64 (作业硬性要求)
epochs	3 (作业硬性要求)
learning_rate	2e-5
batch size	16 (train) / 32 (eval)
weight_decay	0.01
混合精度	bf16 (RTX 4060 支持)
seed	42
训练步数	1719 (9157/16 × 3)
单次耗时	133.7 ~ 219.8 秒

5.2 实验设置

关于模型加载时的键不匹配：加载 BERT 预训练权重时会报告 7 个 UNEXPECTED 键 (cls.predictions.* 的 MLM 头与 cls.seq_relationship.* 的 NSP 头，约 62 万参数) 与 2 个 MISSING 键 (classifier.weight/bias, 仅 2307 个参数)。这是迁移学习预期且必需的行为：预训练任务的头被丢弃，新的分类头被随机初始化后由下游数据训练。若 classifier 不出现 MISSING，反而说明加载了错误的任务头。

5.3 截断分析

由 code/analyze.py 对全部 11447 篇文档统计得到表 7。

表 7: BERT 输入截断统计 (max_length=64, 11447 篇)

项目	值
平均 / 中位数 / P90 / 最长 token 数	834.2 / 872 / 1239 / 6775
被截断的文档占比	99.96%
平均保留 token 比例	7.67%

即 BERT 实际只能看到每篇文档开头约 7.7% 的内容。值得注意的是**中位数 (872)** 大于平均数 (**834.2**)，说明 token 长度分布左偏——少数很短的快讯把均值拉低了，绝大多数文档其实比平均值更长，截断问题比平均值看起来更严重。

图 8 为该统计脚本的完整输出（下半部分为测试集混淆矩阵，见第 6.2 节）。

```

PS C:\Users\jmlcsr\Desktop\lab1> cd c:\Users\jmlcsr\Desktop\lab1; & 'D:\Miniconda\envs\lab1\python.exe' code\
analyze.py
截断统计: 正在 tokenize 11447 篇文档 ...
[transformers] Token indices sequence length is longer than the specified maximum sequence length for this mo
del (1298 > 512). Running this sequence through the model will result in indexing errors

=== 截断统计 (max_length = 64) ===
文档数          : 11447
平均 token 数    : 834.2
中位数 / P90 / 最长 : 872 / 1239 / 6775
超过 64 的文档占比 : 99.96%
平均保留 token 比例 : 7.67%
-> 已写入 results\truncation_stats.json

=== 错误分析 ===
pred      business  politics  sports
gold
business   129       11        2
politics    4       139        0
sports      0        3       857

错误样本数: 20 / 1145
-> 已写入 results\error_analysis.md
PS C:\Users\jmlcsr\Desktop\lab1> 

```

图 8: analyze.py 的输出: 上半部分为截断统计, 下半部分为 BERT 在测试集上的混淆矩阵与错误样本数。

5.4 实验结果

由于 GPU 浮点运算存在非确定性, 同一配置重复运行的结果并不完全相同, 因此本节给出 4 次独立运行的统计结果 (各次明细见表 8)。

表 8: BERT 微调的 4 次独立运行结果

次数	Val Macro-F1 (ep1 / ep2 / ep3)	Test Acc	Test Macro-F1
1	0.9463 / 0.9346 / 0.9512	0.9843	0.9630
2	0.9436 / 0.9440 / 0.9494	0.9817	0.9558
3	0.9401 / 0.9307 / 0.9455	0.9843	0.9620
4	0.9488 / 0.9384 / 0.9431	0.9825	0.9582
均值 ± 标准差	—	0.9832 ± 0.0013	0.9597 ± 0.0033

图 9~ 图 11 为第 2~4 次运行的训练日志 (第 1 次的评测行见第 5.6 节)。

5.5 结果分析

(1)BERT 取得最优结果,但优势有限。4 次运行都优于最强的词袋(Word Frequency, 0.9542), 平均高 +0.0055; 但在 Accuracy 上基本持平 (0.9832 vs 0.9825), 其中第 2 次运行 (0.9817) 甚至略低于词袋。因此准确的表述是“与最强词袋基线相当或略有优势”——“BERT 全面优于传统方法”的直觉在本任务上并不成立。

(2) 只看到 7.7% 的内容, 却拿到了最优的 Macro-F1。原因是这 64 个 token 对应

```

{'loss': '0.5369', 'grad_norm': '3.977', 'learning_rate': '1.943e-05', 'epoch': '0.08726'}
{'loss': '0.1501', 'grad_norm': '0.3715', 'learning_rate': '1.885e-05', 'epoch': '0.1745'}
{'loss': '0.09516', 'grad_norm': '6.645', 'learning_rate': '1.827e-05', 'epoch': '0.2618'}
{'loss': '0.1256', 'grad_norm': '4.097', 'learning_rate': '1.768e-05', 'epoch': '0.349'}
{'loss': '0.08607', 'grad_norm': '2.948', 'learning_rate': '1.71e-05', 'epoch': '0.4363'}
{'loss': '0.1308', 'grad_norm': '55.15', 'learning_rate': '1.652e-05', 'epoch': '0.5236'}
{'loss': '0.0968', 'grad_norm': '7.448', 'learning_rate': '1.594e-05', 'epoch': '0.6108'}
{'loss': '0.0782', 'grad_norm': '4.812', 'learning_rate': '1.536e-05', 'epoch': '0.6981'}
{'loss': '0.1053', 'grad_norm': '4.217', 'learning_rate': '1.478e-05', 'epoch': '0.7853'}
{'loss': '0.06054', 'grad_norm': '8.332', 'learning_rate': '1.419e-05', 'epoch': '0.8726'}
{'loss': '0.07795', 'grad_norm': '3.316', 'learning_rate': '1.361e-05', 'epoch': '0.9599'}
{'eval_loss': '0.1007', 'eval_accuracy': '0.9747', 'eval_macro_f1': '0.9436', 'eval_runtime': '1.03', 'eval_s
amples_per_second': '1112', 'eval_steps_per_second': '34.96', 'epoch': '1'}
{'loss': '0.04794', 'grad_norm': '0.0231', 'learning_rate': '1.303e-05', 'epoch': '1.047'}
{'loss': '0.01918', 'grad_norm': '0.4863', 'learning_rate': '1.245e-05', 'epoch': '1.134'}
{'loss': '0.04138', 'grad_norm': '0.0343', 'learning_rate': '1.187e-05', 'epoch': '1.222'}
{'loss': '0.03976', 'grad_norm': '0.03191', 'learning_rate': '1.129e-05', 'epoch': '1.309'}
{'loss': '0.03454', 'grad_norm': '0.01434', 'learning_rate': '1.07e-05', 'epoch': '1.396'}
{'loss': '0.05827', 'grad_norm': '5.773', 'learning_rate': '1.012e-05', 'epoch': '1.483'}
{'loss': '0.03114', 'grad_norm': '42.24', 'learning_rate': '9.54e-06', 'epoch': '1.571'}
{'loss': '0.03868', 'grad_norm': '0.01169', 'learning_rate': '8.959e-06', 'epoch': '1.658'}
{'loss': '0.047', 'grad_norm': '0.01537', 'learning_rate': '8.377e-06', 'epoch': '1.745'}
{'loss': '0.03632', 'grad_norm': '0.01954', 'learning_rate': '7.795e-06', 'epoch': '1.832'}
{'loss': '0.0497', 'grad_norm': '0.6316', 'learning_rate': '7.213e-06', 'epoch': '1.92'}
{'eval_loss': '0.115', 'eval_accuracy': '0.9747', 'eval_macro_f1': '0.944', 'eval_runtime': '1.006', 'eval_sa
mples_per_second': '1139', 'eval_steps_per_second': '35.8', 'epoch': '2'}
{'loss': '0.01113', 'grad_norm': '0.03448', 'learning_rate': '6.632e-06', 'epoch': '2.007'}
{'loss': '0.02352', 'grad_norm': '0.02027', 'learning_rate': '6.05e-06', 'epoch': '2.094'}
{'loss': '0.006269', 'grad_norm': '5.569', 'learning_rate': '5.468e-06', 'epoch': '2.182'}
{'loss': '0.00263', 'grad_norm': '0.01383', 'learning_rate': '4.887e-06', 'epoch': '2.269'}
{'loss': '0.005778', 'grad_norm': '0.007515', 'learning_rate': '4.305e-06', 'epoch': '2.356'}
{'loss': '0.006394', 'grad_norm': '0.00597', 'learning_rate': '3.723e-06', 'epoch': '2.443'}
{'loss': '0.01494', 'grad_norm': '0.01279', 'learning_rate': '3.141e-06', 'epoch': '2.531'}
{'loss': '0.02366', 'grad_norm': '0.011', 'learning_rate': '2.56e-06', 'epoch': '2.618'}
{'loss': '0.00937', 'grad_norm': '0.004304', 'learning_rate': '1.978e-06', 'epoch': '2.705'}
{'loss': '0.01612', 'grad_norm': '0.008643', 'learning_rate': '1.396e-06', 'epoch': '2.792'}
{'loss': '0.01027', 'grad_norm': '4.733', 'learning_rate': '8.144e-07', 'epoch': '2.88'}
{'loss': '0.01433', 'grad_norm': '0.0277', 'learning_rate': '2.327e-07', 'epoch': '2.967'}
{'eval_loss': '0.1095', 'eval_accuracy': '0.9773', 'eval_macro_f1': '0.9494', 'eval_runtime': '1.016', 'eval_
samples_per_second': '1127', 'eval_steps_per_second': '35.45', 'epoch': '3'}
{'train_runtime': '219.8', 'train_samples_per_second': '125', 'train_steps_per_second': '7.82', 'train_loss':
'0.06213', 'epoch': '3'}
100% | 1719/1719 [03:39<00:00, 7.82it/s]

```

图 9: BERT 第 2 次运行的训练日志。1,719 步，三轮 Val Macro-F1 依次为 0.9436 / 0.9440 / 0.9494。

```

{'loss': '0.5598', 'grad_norm': '4.69', 'learning_rate': '1.943e-05', 'epoch': '0.08726'}
{'loss': '0.1663', 'grad_norm': '0.3947', 'learning_rate': '1.885e-05', 'epoch': '0.1745'}
{'loss': '0.1027', 'grad_norm': '3.012', 'learning_rate': '1.827e-05', 'epoch': '0.2618'}
{'loss': '0.1191', 'grad_norm': '6.476', 'learning_rate': '1.768e-05', 'epoch': '0.349'}
{'loss': '0.08229', 'grad_norm': '1.475', 'learning_rate': '1.71e-05', 'epoch': '0.4363'}
{'loss': '0.1337', 'grad_norm': '124.7', 'learning_rate': '1.652e-05', 'epoch': '0.5236'}
{'loss': '0.1113', 'grad_norm': '4.603', 'learning_rate': '1.594e-05', 'epoch': '0.6108'}
{'loss': '0.09021', 'grad_norm': '3.124', 'learning_rate': '1.536e-05', 'epoch': '0.6981'}
{'loss': '0.1069', 'grad_norm': '2.564', 'learning_rate': '1.478e-05', 'epoch': '0.7853'}
{'loss': '0.06156', 'grad_norm': '7.931', 'learning_rate': '1.419e-05', 'epoch': '0.8726'}
{'loss': '0.0755', 'grad_norm': '3.001', 'learning_rate': '1.361e-05', 'epoch': '0.9599'}
{'eval_loss': '0.0996', 'eval_accuracy': '0.9729', 'eval_macro_f1': '0.9401', 'eval_runtime': '0.7668', 'eval_samples_per_second': '1493', 'eval_steps_per_second': '46.95', 'epoch': '1'}
{'loss': '0.0487', 'grad_norm': '0.01826', 'learning_rate': '1.303e-05', 'epoch': '1.047'}
{'loss': '0.02929', 'grad_norm': '0.151', 'learning_rate': '1.245e-05', 'epoch': '1.134'}
{'loss': '0.03254', 'grad_norm': '0.02507', 'learning_rate': '1.187e-05', 'epoch': '1.222'}
{'loss': '0.05952', 'grad_norm': '0.03092', 'learning_rate': '1.129e-05', 'epoch': '1.309'}
{'loss': '0.04723', 'grad_norm': '0.03337', 'learning_rate': '1.07e-05', 'epoch': '1.396'}
{'loss': '0.05568', 'grad_norm': '6.963', 'learning_rate': '1.012e-05', 'epoch': '1.483'}
{'loss': '0.04209', 'grad_norm': '38.63', 'learning_rate': '9.54e-06', 'epoch': '1.571'}
{'loss': '0.03416', 'grad_norm': '0.01514', 'learning_rate': '8.959e-06', 'epoch': '1.658'}
{'loss': '0.04713', 'grad_norm': '0.01508', 'learning_rate': '8.377e-06', 'epoch': '1.745'}
{'loss': '0.04212', 'grad_norm': '0.02601', 'learning_rate': '7.795e-06', 'epoch': '1.832'}
{'loss': '0.04879', 'grad_norm': '0.2192', 'learning_rate': '7.213e-06', 'epoch': '1.92'}
{'eval_loss': '0.1381', 'eval_accuracy': '0.9703', 'eval_macro_f1': '0.9307', 'eval_runtime': '0.7622', 'eval_samples_per_second': '1502', 'eval_steps_per_second': '47.23', 'epoch': '2'}
{'loss': '0.01323', 'grad_norm': '1.281', 'learning_rate': '6.632e-06', 'epoch': '2.007'}
{'loss': '0.02508', 'grad_norm': '0.03151', 'learning_rate': '6.05e-06', 'epoch': '2.094'}
{'loss': '0.01986', 'grad_norm': '6.794', 'learning_rate': '5.468e-06', 'epoch': '2.182'}
{'loss': '0.01456', 'grad_norm': '0.02792', 'learning_rate': '4.887e-06', 'epoch': '2.269'}
{'loss': '0.01425', 'grad_norm': '0.006945', 'learning_rate': '4.305e-06', 'epoch': '2.356'}
{'loss': '0.01312', 'grad_norm': '0.005956', 'learning_rate': '3.723e-06', 'epoch': '2.443'}
{'loss': '0.01971', 'grad_norm': '0.008877', 'learning_rate': '3.141e-06', 'epoch': '2.531'}
{'loss': '0.02514', 'grad_norm': '0.02003', 'learning_rate': '2.56e-06', 'epoch': '2.618'}
{'loss': '0.01376', 'grad_norm': '0.003694', 'learning_rate': '1.978e-06', 'epoch': '2.705'}
{'loss': '0.01012', 'grad_norm': '0.008645', 'learning_rate': '1.396e-06', 'epoch': '2.792'}
{'loss': '0.007327', 'grad_norm': '4.61', 'learning_rate': '8.144e-07', 'epoch': '2.88'}
{'loss': '0.006609', 'grad_norm': '0.02521', 'learning_rate': '2.327e-07', 'epoch': '2.967'}
{'eval_loss': '0.1133', 'eval_accuracy': '0.9755', 'eval_macro_f1': '0.9455', 'eval_runtime': '0.7371', 'eval_samples_per_second': '1553', 'eval_steps_per_second': '48.84', 'epoch': '3'}
{'train_runtime': '136.4', 'train_samples_per_second': '201.4', 'train_steps_per_second': '12.61', 'train_loss': '0.06647', 'epoch': '3'}
100% | 1719/1719 [02:16<00:00, 12.61it/s]

```

图 10: BERT 第 3 次运行的训练日志。三轮 Val Macro-F1 依次为 0.9401 / 0.9307 / 0.9455。

```

{'loss': '0.5659', 'grad_norm': '5.696', 'learning_rate': '1.943e-05', 'epoch': '0.08726'}
{'loss': '0.1634', 'grad_norm': '0.4226', 'learning_rate': '1.885e-05', 'epoch': '0.1745'}
{'loss': '0.1063', 'grad_norm': '5.397', 'learning_rate': '1.827e-05', 'epoch': '0.2618'}
{'loss': '0.1183', 'grad_norm': '3.13', 'learning_rate': '1.768e-05', 'epoch': '0.349'}
{'loss': '0.07947', 'grad_norm': '3.716', 'learning_rate': '1.71e-05', 'epoch': '0.4363'}
{'loss': '0.119', 'grad_norm': '5.213', 'learning_rate': '1.652e-05', 'epoch': '0.5236'}
{'loss': '0.1122', 'grad_norm': '5.055', 'learning_rate': '1.594e-05', 'epoch': '0.6108'}
{'loss': '0.08993', 'grad_norm': '3.003', 'learning_rate': '1.536e-05', 'epoch': '0.6981'}
{'loss': '0.1081', 'grad_norm': '4.874', 'learning_rate': '1.478e-05', 'epoch': '0.7853'}
{'loss': '0.06103', 'grad_norm': '5.541', 'learning_rate': '1.419e-05', 'epoch': '0.8726'}
{'loss': '0.08058', 'grad_norm': '4.051', 'learning_rate': '1.361e-05', 'epoch': '0.9599'}
{'eval_loss': '0.09494', 'eval_accuracy': '0.9764', 'eval_macro_f1': '0.9488', 'eval_runtime': '0.7419', 'eval_samples_per_second': '1543', 'eval_steps_per_second': '48.52', 'epoch': '1'}
{'loss': '0.04507', 'grad_norm': '0.01323', 'learning_rate': '1.303e-05', 'epoch': '1.047'}
{'loss': '0.03582', 'grad_norm': '0.08223', 'learning_rate': '1.245e-05', 'epoch': '1.134'}
{'loss': '0.04941', 'grad_norm': '0.02694', 'learning_rate': '1.187e-05', 'epoch': '1.222'}
{'loss': '0.05671', 'grad_norm': '0.0191', 'learning_rate': '1.129e-05', 'epoch': '1.309'}
{'loss': '0.04048', 'grad_norm': '0.02089', 'learning_rate': '1.07e-05', 'epoch': '1.396'}
{'loss': '0.05445', 'grad_norm': '5.312', 'learning_rate': '1.012e-05', 'epoch': '1.483'}
{'loss': '0.04999', 'grad_norm': '13.16', 'learning_rate': '9.54e-06', 'epoch': '1.571'}
{'loss': '0.04149', 'grad_norm': '0.01583', 'learning_rate': '8.959e-06', 'epoch': '1.658'}
{'loss': '0.04853', 'grad_norm': '0.01383', 'learning_rate': '8.377e-06', 'epoch': '1.745'}
{'loss': '0.04025', 'grad_norm': '0.0201', 'learning_rate': '7.795e-06', 'epoch': '1.832'}
{'loss': '0.04793', 'grad_norm': '0.2784', 'learning_rate': '7.213e-06', 'epoch': '1.92'}
{'eval_loss': '0.1216', 'eval_accuracy': '0.9729', 'eval_macro_f1': '0.9384', 'eval_runtime': '0.7671', 'eval_samples_per_second': '1493', 'eval_steps_per_second': '46.93', 'epoch': '2'}
{'loss': '0.01401', 'grad_norm': '0.01591', 'learning_rate': '6.632e-06', 'epoch': '2.007'}
{'loss': '0.02263', 'grad_norm': '0.03958', 'learning_rate': '6.05e-06', 'epoch': '2.094'}
{'loss': '0.01306', 'grad_norm': '6.573', 'learning_rate': '5.468e-06', 'epoch': '2.182'}
{'loss': '0.006915', 'grad_norm': '0.02305', 'learning_rate': '4.887e-06', 'epoch': '2.269'}
{'loss': '0.003998', 'grad_norm': '0.006423', 'learning_rate': '4.305e-06', 'epoch': '2.356'}
{'loss': '0.005942', 'grad_norm': '0.005046', 'learning_rate': '3.723e-06', 'epoch': '2.443'}
{'loss': '0.01411', 'grad_norm': '0.007257', 'learning_rate': '3.141e-06', 'epoch': '2.531'}
{'loss': '0.02571', 'grad_norm': '0.02228', 'learning_rate': '2.56e-06', 'epoch': '2.618'}
{'loss': '0.01029', 'grad_norm': '0.004099', 'learning_rate': '1.978e-06', 'epoch': '2.705'}
{'loss': '0.009757', 'grad_norm': '0.006896', 'learning_rate': '1.396e-06', 'epoch': '2.792'}
{'loss': '0.01867', 'grad_norm': '4.768', 'learning_rate': '8.144e-07', 'epoch': '2.88'}
{'loss': '0.01131', 'grad_norm': '0.01636', 'learning_rate': '2.327e-07', 'epoch': '2.967'}
{'eval_loss': '0.1207', 'eval_accuracy': '0.9747', 'eval_macro_f1': '0.9431', 'eval_runtime': '0.7419', 'eval_samples_per_second': '1543', 'eval_steps_per_second': '48.52', 'epoch': '3'}
{'train_runtime': '133.7', 'train_samples_per_second': '205.4', 'train_steps_per_second': '12.86', 'train_loss': '0.06622', 'epoch': '3'}

```

图 11: BERT 第 4 次运行的训练日志。三轮 Val Macro-F1 依次为 0.9488 / 0.9384 / 0.9431。

的是**标题 + 导语**，而新闻导语恰恰是主题信息最密集的部分。这说明“截断”不等于“丢信息”，**关键看截掉了什么**。但截断同时也限制了 BERT 拉开差距的空间：它用 8% 的内容只换来对“用满 100% 词汇”的词袋的微弱优势。

(3) 轮间波动明显。例如第 1 次运行的三轮 Val Macro-F1 为 0.9463 / 0.9346 / 0.9512，第 4 次为 0.9488 / 0.9384 / 0.9431。epoch 1 到 2 之间时常不升反降，与学习率在前期衰减（线性衰减调度）的过程相符。4 次运行的末轮都接近各自最优，但不能假设末轮总是最优。

5.6 多次运行与可复现性

问题：同一配置（seed=42）、同一数据划分，重复运行得到的指标并不相同（Test Macro-F1 从 0.9558 到 0.9630，极差 0.0072）。

证据：第 1 次与第 2 次运行在**第 50 步**的 train_loss 就已经不同（0.5087 vs 0.5369）。而第 50 步之前两次运行都没有写过任何 **checkpoint**，故该差异**不是** checkpoint 保存/回载造成的，而是训练过程本身的分歧。

第 1 次运行的三轮验证指标（节选关键字段）：

```
{'eval_loss': 0.0916, 'eval_accuracy': 0.9755, 'eval_macro_f1': 0.9463, 'epoch': '1'}
{'eval_loss': 0.1230, 'eval_accuracy': 0.9712, 'eval_macro_f1': 0.9346, 'epoch': '2'}
{'eval_loss': 0.1121, 'eval_accuracy': 0.9782, 'eval_macro_f1': 0.9512, 'epoch': '3'}
```

原因：训练在 GPU 上进行，存在非确定性 kernel。最主要的是 nn.Embedding 的反向传播使用 atomicAdd 把梯度累加到同一个词向量上（浮点加法顺序不定），其次是 scaled_dot_product_attention 的反向 kernel，以及 bf16 下的归约顺序。seed=42 只能固定**参数初始化与数据顺序**，无法约束这些 kernel 的执行顺序。

处理：报告 BERT 时给出均值 ± 标准差而非单次值；结论按“相当或略有优势”表述，不下“显著优于”的强结论。作为反证对照，Task 1 已实测重跑逐位一致（确定性），Task 2 的 gensim 在固定 seed 后亦稳定，即不确定性只出现在 GPU 上的 BERT 微调。

6 综合对比与讨论

6.1 三种文本表示的横向对比

按 Test Macro-F1 的三层结论：

(1) 词袋在这个数据集上异常强（0.9542）。文本极长（平均 637 词），词袋能无损保留全部词汇信息；而“平均词向量”把 637 词压成 100 维，信息损失巨大。这与“词向量必然优于词袋”的常见直觉相反，是**长文本 + 类别可分性强**这一数据特点造成的。

(2) 词向量的价值体现在“压缩比”而非“绝对性能”。100 维追平 53146 维，维度降低约 500 倍而性能基本不变。

表 9: 全部方法在 NYT Test Set (1145 条) 上的结果汇总

方法	表示类型	维度	Test Accuracy	Test Macro-F1
多数类基线 (全预测 sports)	—	—	0.7511	0.2860
Binary BoW + LR	二值词袋	53146	0.9790	0.9468
Word Frequency + LR	词频词袋	53146	0.9825	0.9542
Binary BoW + LR (balanced)	二值词袋	53146	0.9790	0.9479
Word Frequency + LR (balanced)	词频词袋	53146	0.9808	0.9506
GloVe-100 平均 + LR	预训练词向量	100	0.9782	0.9471
Word2Vec(NYT)-100 平均 + LR	自训练词向量	100	0.9712	0.9308
Word2Vec(AG News)-100 平均 + LR	自训练词向量	100	0.9686	0.9227
BERT-base-uncased	预训练语言模型微调	768	0.9832	0.9597

(3) 只有端到端的微调才超过词袋, 但优势很小 (平均 +0.0055, 约为运行波动 0.0033 的 1.7 倍)。在长文本且类别可分性强的数据上, 词袋本身就是极强的基线, 加上 BERT 被硬性限制在 64 token, 它很难拉开差距。

6.2 错误分析

以第 4 次运行的预测 (Test Acc 0.9825) 为例, 错误共 **20 / 1145 (1.75%)**。

表 10: BERT 测试集混淆矩阵 (行 = 真实, 列 = 预测)

真实 \ 预测	business	politics	sports	错分率
business (142)	129	11	2	9.15%
politics (143)	4	139	0	2.80%
sports (860)	0	3	857	0.35%

由混淆矩阵可精确复原各类别指标 (与 results.json 中的 Macro-F1 0.9582 完全一致):

表 11: 由混淆矩阵计算的各类别指标 (第 4 次运行)

类别	Precision	Recall	F1
business	0.9699	0.9085	0.9382
politics	0.9085	0.9720	0.9392
sports	0.9977	0.9965	0.9971
Macro 平均	0.9587	0.9590	0.9582

错误分布高度集中:

表 12: 错误样本的类别对分布

真实 → 预测	次数	占全部错误
business → politics	11	55%
politics → business	4	20%
sports → politics	3	15%
business → sports	2	10%

结论:

(1) 错误集中在 **business ↔ politics** ($15/20 = 75\%$)。两类都是“社会经济/公共事务”题材，边界天然模糊。典型错例：“*this spring, the missouri chamber of commerce urged the state legislature to accept the federal government’s plan to expand medicaid for the poor and disabled. the business lobbying group had not suddenly gone rogue.*”——商业团体在做政治游说，被判定为 politics 并不难理解。

(2) **sports** 几乎不会错（错分率 0.35%，3/860）。该类的判别特征极强，也是它支撑起 0.98 Accuracy 的原因。

(3) 错误方向不对称：business → politics 有 11 次，反向仅 4 次。business 报道常涉及政策与政府，更容易被“政治化”。

(4) 存在标注歧义的可能。错例中包含“*the departure of chairman ben s. bernanke of the federal reserve*”、“*the costs of the 16-day partial government shutdown*”等本身横跨政治与经济的报道，人工标注也有争议空间。

(5) **Accuracy** 的钝感。sports 占 75.1%，即使把 13 个 business 错例全部修正，Accuracy 也只提升约 0.011。这再次说明在该数据集上 **Macro-F1** 比 **Accuracy** 更有区分度。

6.3 结论与局限性

主要结论

1. 在 NYT 三分类任务上，词袋表示是一个极强的基线，Word Frequency + LR 即可达到 0.9542 的 Macro-F1；
2. 词向量平均的主要贡献是维度压缩（约 500 倍）而非绝对性能提升，且其性能与词覆盖率完全同序；
3. **BERT** 微调取得最优 **Macro-F1** (0.9597 ± 0.0033)，但相对词袋的优势很小（平均 +0.0055），且单次结果不可复现；
4. 错误集中在语义边界模糊的类对（business ↔ politics），而多数类 sports 几乎不会错。

局限性

- 文档向量取平均丢失了语序，这是 Task 2 性能受限的重要内因；
- BERT 受 64 token 截断限制，只使用约 7.7% 的文本，若能放宽长度（如 256），其优势可能更明显；
- GPU 非确定性使得 BERT 的结果需以多次运行的均值表述，本次仅运行 4 次，标准差估计本身也不够稳定；
- 受算力限制，未进行超参数搜索（学习率、batch size、warmup 均取常用默认值），warmup_ratio 因所用 transformers 版本不支持而被忽略。

A 运行环境与复现步骤

完整代码与运行说明见仓库根目录的 README.md，本实验记录见 results/实验记录.md，机器可读指标见 results/results.json。主要运行顺序如下（需先安装依赖并下载 nltk 分词数据与 GloVe 词向量）：

```
# Step 0: 统一数据划分 (80/10/10, seed=42)
```

```
python code/step0_split.py --stratify
```

```
# Task 1: 两种词袋表示
```

```
python code/task1_bow.py --mode binary
```

```
python code/task1_bow.py --mode freq
```

```
# Task 2: 三组词向量实验
```

```
python code/task2_word2vec.py --source glove --glove embeddings/glove.6B.100d.txt
```

```
python code/task2_word2vec.py --source ag
```

```
python code/task2_word2vec.py --source nyt
```

```
# Task 3: BERT 微调 (3 epochs)
```

```
python code/task3_bert.py --no_save --epochs 3
```

```
# 收尾分析：截断统计 + 错误分析
```

```
python code/analyze.py
```

其中 `--no_save` 表示不保存 checkpoint、训练结束后不回载“最佳模型”。这样处理有两个目的：一是使最终评测严格对应第 3 轮结束时的模型，与作业“训练 3 epochs”的要求严格对应；二是规避 transformers 5.19 在 `save_pretrained` $\rightarrow$ `load_state_dict` 往返中 LayerNorm 参数命名不一致（存为 `gamma/beta`、读取时期望 `weight/bias`）所产生的大量警告（详见附录 B）。

B 问题排查记录

本节记录实现过程中遇到并定位的四个问题，其中第 4 条是一个由自己引入的真实 bug。

(1) **sklearn 版本变化导致的参数失效警告**。训练 Task 1 时 sklearn 1.9 报出 FutureWarning: 'n_jobs' has no effect since 1.8 and will be removed in 1.10 (见图 12)。该参数在新版本已被忽略，为保持输出干净并避免未来版本报错，已将其移除；移除后重跑结果逐位一致，反过来也验证了 Task 1 的确定性。

```

PS C:\Users\jmlcsr\Desktop\lab1> & 'D:\Miniconda\envs\lab1\python.exe' code\task1_bow.py --mode binary
[binary] 词表大小 |V| = 53146, 训练集矩阵维度 = (9157, 53146)
[binary] 稀疏矩阵非零元素占比 = 0.005935
D:\Miniconda\envs\lab1\Lib\site-packages\sklearn\linear_model\_logistic.py:1435: FutureWarning: 'n_jobs' has
no effect since 1.8 and will be removed in 1.10. You provided 'n_jobs=-1', please leave it unspecified.
  warnings.warn(msg, category=FutureWarning)
[多数类基线(全预测 sports)] Accuracy=0.7511 Macro-F1=0.2860
[Binary-Bow + LR (val)] Accuracy=0.9773 Macro-F1=0.9436
      precision    recall  f1-score   support

   business      0.9394      0.8794      0.9084        141
   politics      0.9067      0.9444      0.9252        144
    sports      0.9954      0.9988      0.9971        860

   accuracy                   0.9773        1145
  macro avg      0.9471      0.9409      0.9436        1145
 weighted avg      0.9773      0.9773      0.9771        1145

[Binary-Bow + LR] Accuracy=0.9790 Macro-F1=0.9468
      precision    recall  f1-score   support

   business      0.9545      0.8873      0.9197        142
   politics      0.8896      0.9580      0.9226        143
    sports      0.9988      0.9977      0.9983        860

   accuracy                   0.9790        1145
  macro avg      0.9477      0.9477      0.9468        1145
 weighted avg      0.9797      0.9790      0.9791        1145

结果已记录到 results\results.json
PS C:\Users\jmlcsr\Desktop\lab1> & 'D:\Miniconda\envs\lab1\python.exe' code\task1_bow.py --mode freq

```

图 12: Binary BoW 运行时 sklearn 打印的 n_jobs 参数失效警告，其后为 Word Frequency 的运行命令。该参数随后被移除。

(2) **transformers 5.19 的参数改名与不支持**。TrainingArguments 中 evaluation_strategy 已在 4.46 改名为 eval_strategy; warmup_ratio 在当前版本不被支持。脚本通过 inspect.signature 动态检查可用参数名，自动选择正确的名字并过滤掉不支持的参数（运行时会打印 [warn] 当前 transformers 版本不支持这些参数，已忽略: ['warmup_ratio']），因此实际训练未使用 warmup。

(3) **LayerNorm 参数在保存/加载往返中的键不匹配**。不使用 --no_save 时，训练结束回载最佳 checkpoint 会打印约 50 个键的 missing/unexpected 警告：保存的键名是...LayerNorm.gamma/beta，而加载时期望...LayerNorm.weight/bias。为确认其是否影响结果，对比了回载前后的验证指标：

回载前（训练中评测）：eval_loss 0.1121 eval_accuracy 0.9782 eval_macro_f1 0.9512

```
回载后 (trainer.evaluate): eval_loss 0.11212880... eval_accuracy 0.97816593...
                             eval_macro_f1 0.95117597...
```

两者逐位一致，说明该警告不影响最终结果，属于命名往返问题。正式实验改用 `--no_save` 后该路径不再触发。

(4) 一个真实的 bug：报错点在 `collator`，根因在数据构造。加入 `--no_save` 选项后，训练在第一步即抛出：

```
TypeError: vars() argument must have __dict__ attribute
(transformers/data/data_collator.py:126)
```

最初怀疑是 `collator` 被框架重复包装，但在换成自定义 `collator` 后，报错变成 `AttributeError: 'TextClfDataset' object has no attribute 'keys'`——两次报错都指向“`collate_fn` 收到的第一个元素就是数据集对象本身”。据此回看数据构造代码，发现该行末尾多了一个逗号：

```
ds_train = TextClfDataset(train_df["text"], y_tr, tokenizer), args.no_save
```

尾随逗号使右值成为 `tuple`，`torch` 便把该 `tuple` 当作“数据集”：其长度为 2，`ds[0]` 恰好是那个 `TextClfDataset` 对象，于是 `collate_fn` 收到 `[Dataset]`，默认 `collator` 的 `vars(f)` 与自定义 `collator` 的 `f.keys()` 相继失败。修正该行后，`--no_save` 连续 3 次稳定运行成功。这里的体会是：AI 大规模改动代码时容易在不经意处引入小错误，而这类错误让 AI 反复推理往往定位不到，最终是靠让它把相关代码完整遍历一遍发现了根因。

附：GloVe 词向量获取过程中的网络问题。官方地址 <http://nlp.stanford.edu/data/glove.6B.zip>（约 822 MB）在国内直连被限速到约 12 KB/s；改用 Python 经系统代理访问镜像后成功下载（331 MiB，约 1 分钟，~5.3 MB/s）。经验：换源前必须核对文件大小（正确为 347,117,594 字节，某些镜像提供的是仅 134 MB 的裁剪版本）；且 Windows 自带的 `curl.exe` 不读取系统代理，而 Python 的 `urllib` 会读取。